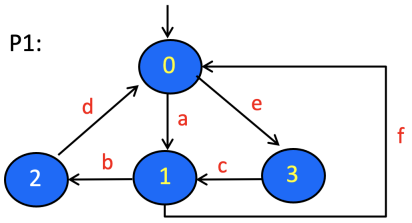
# Exercises Testing: McCabe and EFSM

## 1 McCabe-cyclomatic-based Testing

1. Let $C = \{c_1, c_2, c_3\}$ be a set of linearly independent circuits in a CFG. What does it mean for these circuits to be linearly independent from each other?

2. Consider the CFG $P_1$ shown below, with 0 as the entry node, and 1 as the exit node. The edges have been labelled with ids: $a...e$.

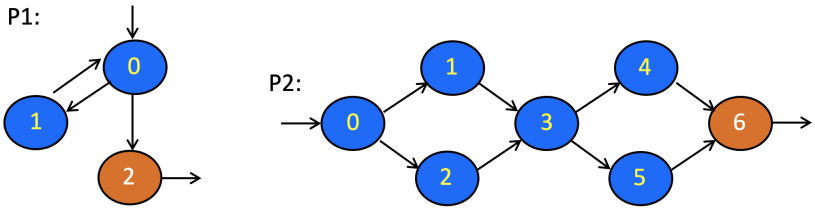



   (a) Consider the circuit $[0, 1, 2, 0]$. How to represent this circuit as a vector over the edges of the graph?

   (b) Give five other circuits in $P_1$. Give few example of circuits which are linearly independent from each other, and few examples which are not.

   (c) What is the McCabe cyclomatic number of $P_1$? Which circuits of $P_1$ should be included in your test requirement (TR) according to McCabe?

3. Consider programs $P_1$ and $P_2$ with the CFGs shown below. Node 0 is the entry node. Orange node is the exit node.




   (a) What are the McCabe cyclomatic numbers of those programs?

   (b) Give a set $C$ of linearly independent 'circuits' for each of the program, after adding a fake back-edge from its exit node to go back to the entry node. Try to find a $C$ that is *maximal* (containing as many circuits as possible).

   (c) Which paths should be in the test requirement (TR) of $P_1$ and $P_2$, according to McCabe

   (d) Give a minimum sized test-set that gives full McCabe coverage for each program (covering all paths in the TR in your answer in (c)).

## 2 EFSM

1. Consider a simplified course management system. The system manages a single course. Students can register to the course through the system. Actions possible on the system:

   - $add(student)$ to add a student to the course. This is only possible if the number of students already enrolled to the course is below $Max$. Else the student is added to the course's waiting list.
   - $remove(student)$ removes a student from the course and its waiting list. If the removed student was not in the waiting list, a student in the waiting list will be enrolled into the course, if the waiting list was not empty.

Initially, the course is empty (no student is enrolled), and its waiting list is of course also empty.

Your tasks:

(a) Propose an EFSM (extended finite state machine) model for the above system. For testing purposes, you can assume that you can inspect variables $C$ and $W$ representing, respectively, the set of students that are currently enrolled to course, and the set of students currently in the waiting list.

The post-conditions in the above model are partial. For example, $add(s)$ from state 0 back to state 0 only asserts $s \in C$ as the post-condition. To be more complete you can use $C = \mathbf{old}(C)/\{s\}$ as the post-condition. This would spot a bug where *add* removes a student from the course.

(b) Give *positive* tests that would cover all prime paths in your model. What should be tested?

(c) Give *negative* tests that would cover cover all transitions in the reject-state-extended model. What should be tested in these tests?

2. Imagine an ATM system.


Let's consider the following simplified version of such a system. The user can interact with it using the following actions:

- $insert(C)$ : for the user to insert his/her bank-card $C$ to the ATM.
- $pin(n)$ : for the user to enter a pincode $n$ for his/her card.
- $withdraw(amount)$ : for the user to withdraw cash from the ATM. Only possible if the user has enough in its card balance and the ATM has enough cash. Only a non-negative $n$ can physically be entered, so you don't necessarily need to check that.
- $done()$ : to indicate that the user has finished using the ATM.

The following actions are actions of the ATM:

- $eject()$ : to eject the card that is currently in the ATM.
- $givecash(m)$ : to give the user cash of amount $m$.

For testing purposes, let's assume the following variables are observable to inspect the ATM's state:

- $cash$ : the amount of cash the ATM has left.
- $card$ : the card that is currently in the ATM. Null is there is none.
- $balance$ : the card's current balance (amount of money the card owner has left in his/her card/bank).
- Additionally, you have access to the predicate $valid(n, C)$ that gives true if $n$ is a valid pincode for the card $C$, and else false.

Your tasks:

(a) Propose an EFSM model for the above ATM system.

(b) Give a set of *positive* tests with full prime path coverage on your model. What should be tested?

(c) Give a set of *negative* tests that cover all prime paths that in the reject-state extended model, that include the transition $4 \rightarrow 3$ (the ATM gives cash) in the original model. What should be tested in these tests?